



Full Name: Ahmed Elgohary
Email: gohary200023@gmail.com
Test Name: Mock Test
Taken On: 22 Sep 2025 18:36:32 IST
Time Taken: 14 min 59 sec/ 15 min
Invited by: Ankush
Invited on: 22 Sep 2025 18:32:58 IST
Skills Score:
Tags Score:

- Algorithms 0/105
- Core CS 0/105
- Easy 0/105
- Problem Solving 0/105
- Search 0/105
- Sorting 0/105
- problem-solving 0/105

0%
0/105

scored in **Mock Test** in 14 min 59 sec on 22 Sep 2025 18:36:32 IST

Recruiter/Team Comments:

No Comments.

	Question Description	Time Taken	Score	Status
Q1	Find the Median > Coding	14 min 54 sec	0/ 105	

QUESTION 1

Wrong Answer

Score 0

Find the Median > Coding

SortingSearchAlgorithmsEasyproblem-solvingCore CS

Problem Solving

QUESTION DESCRIPTION

The median of a list of numbers is essentially its middle element after sorting. The same number of elements occur after it as before. Given a list of numbers with an odd number of elements, find the **median**?

Example
 $arr = [5, 3, 1, 2, 4]$

The sorted array $arr' = [1, 2, 3, 4, 5]$. The middle element and the median is **3**.

Function Description

Complete the `findMedian` function in the editor below.

`findMedian` has the following parameter(s):

• `int arr[n]`: an unsorted array of integers

Returns

- `int`: the median of the array

Input Format

The first line contains the integer n , the size of `arr`.

The second line contains n space-separated integers `arr[i]`

Constraints

- $1 \leq n \leq 1000001$
- n is odd
- $-10000 \leq arr[i] \leq 10000$

Sample Input 0

```
7
0 1 2 4 6 5 3
```

Sample Output 0

```
3
```

Explanation 0

The sorted `arr` = `[0, 1, 2, 3, 4, 5, 6]`. It's middle element is at `arr[3] = 3`.

CANDIDATE ANSWER

Language used: C#

```
1
2 class Result
3 {
4
5     /*
6      * Complete the 'findMedian' function below.
7      *
8      * The function is expected to return an INTEGER.
9      * The function accepts INTEGER_ARRAY arr as parameter.
10     */
11
12     public static int findMedian(List<int> arr)
13     {
14         int localMin = arr[0];
15         List<int> tempArr = new();
16         int x = 0;
17         while(true)
18         {
19             for(int i = 0; i< arr.Count-1; i++){
20                 if(arr[i]<arr[i+1]){
21                     localMin = arr[i];
22                 }
23                 else localMin = arr[i+1];
24             }
25
26             tempArr.Add(localMin);
27             arr.Remove(localMin);
28
29             if(arr.Count == 0)
30                 break;
```

```
31     }
32
33     return (tempArr[ ((tempArr.Count-1)/2)+1]);
34 }
35
36 }
37
```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 1	Easy	Sample case	✘ Abort Called	0	0.7075 sec	250 KB
Testcase 2	Easy	Hidden case	✘ Abort Called	0	1.1749 sec	251 KB
Testcase 3	Easy	Hidden case	✘ Abort Called	0	1.5578 sec	268 KB
Testcase 4	Easy	Hidden case	✘ Terminated due to timeout	0	3.0068 sec	34.1 KB

No Comments